LAB 04 EVIDENCE KELYAN IHINACK

Builds and Five-Environment Promotion Pipeline. Path B (local harness). Repo: github.com/kelyanzzz/lab04-citybike-sdlc

Mission 0

Setup baseline: bare repo created, images pulled, starter cloned. *npm run build* creates the manifest and /health answers ok. node_modules is 139 MB.



The 10 devDependencies: @babel/core, eslint, jest, nodemon, prettier, sass, ts-node, typescript, webpack, webpack-cli.

structuredClone() added in server.js, it works on my machine (Node 20). Commit pushed to the bare repo.



Mission 1

Same code on two clean containers: node:16 crashes with a ReferenceError, node:20 starts.



- A) What does the crash prove? The code only works on my machine. structuredClone() exists from Node 17.
- B) Three other differences: environment variables (e.g. DB_URL), OS or CPU, dependency versions.
- C) Which stage catches it? Build and Staging: CI builds with a fixed Node version and starts it on a clean machine.

Mission 2

Naive Dockerfile (FROM node:20, COPY . ., npm ci, build). The image is 2.04 GB.

```
real 68.798s
>>> docker images citybike
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
IMAGE           ID            DISK USAGE   CONTENT SIZE   EXTRA
citybike:naive  9698f17a9724       2.04GB          494MB
```

After a one-line change in public/, the rebuild runs npm ci again (117 s). The app still answers.

```
#9 unpacking to docker.io/library/citybike:naive 5.3s done
#9 DONE 14.8s
real 117.347s
>>> docker run --rm -p 4173:4173 citybike:naive
CityBike service listening on http://0.0.0.0:4173 [env=development]
>>> curl localhost:4173/health
{"ok":true,"environment":"development","hasBuildManifest":true,"secureKeyConfigured":false}
cat: /proc/996/winpid: No such file or directory
^C
>>> git add . && git commit -m 'Dockerfile: naive single-stage' && git push
[main 301db61] Dockerfile: naive single-stage
 1 file changed, 7 insertions(+)
 create mode 100644 Dockerfile
To C:/Users/kelya/lab/remote.git
   af9dc4d..301db61  main -> main
* Terminal will be reused by tasks, press any key to close it.
```

Build	First build	Rebuild after 1-line change	Size
Naive	69 s	118 s	2.04 GB

- A) COPY . . is before npm ci, so any file change breaks the cache for all the next layers.
- B) Size: the node:20 base image, the devDependencies in node_modules, the files copied from my machine.
- C) Files that should not be there: .git, my local node_modules, dist/, README.md (no .dockerignore).

Mission 3

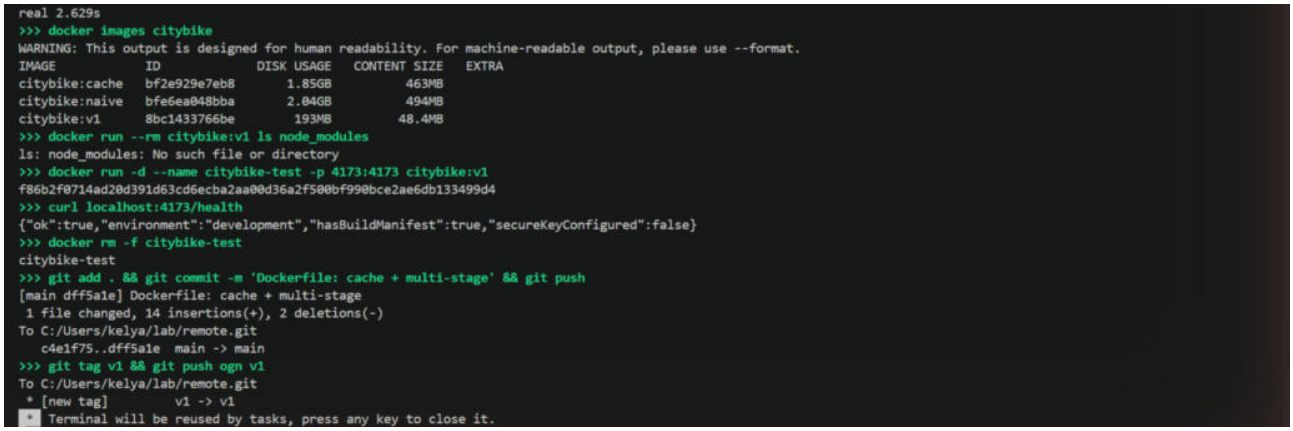
Step 1, cache fix: package files and npm ci first, then the source, plus a .dockerignore. npm ci is CACHED, the rebuild takes 2.3 s.

```
#8 [4/6] RUN npm ci
#8 CACHED
real 2.274s
>>> docker images citybike
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
IMAGE           ID            DISK USAGE   CONTENT SIZE   EXTRA
citybike:cache  bf2e929e7eb8       1.85GB          463MB
citybike:naive  bfe6ea048bba       2.04GB          494MB
>>> git add . && git commit -m 'Dockerfile: cache-friendly layer order + .dockerignore' && git push
[main c4e1f75] Dockerfile: cache-friendly layer order + .dockerignore
 2 files changed, 25 insertions(+), 2 deletions(-)
 create mode 100644 .dockerignore
To C:/Users/kelya/lab/remote.git
   301db61..c4e1f75  main -> main
```

Step 2, shortcut npm ci --omit=dev: BUILD ERROR. npm run build needs TypeScript, a devDependency, at build time.

```
#10 0.497 BUILD ERROR: Necessary devDependencies missing. Run npm run build in the builder stage.
#10 ERROR: process "/bin/sh -c npm run build" did not complete successfully: exit code: 1
------
 > [6/6] RUN npm run build:
0.462
0.462 > citybike-sdlc@1.0.1 build
0.462 > node scripts/build-manifest.js
0.462
0.497 BUILD ERROR: Necessary devDependencies missing. Run npm run build in the builder stage.
------
Dockerfile:7
--------------------
   5 |     RUN npm ci --omit=dev
   6 |     COPY . .
   7 | >>> RUN npm run build
   8 |     EXPOSE 4173
   9 |     CMD ["node", "server.js"]
--------------------
ERROR: failed to build: failed to solve: process "/bin/sh -c npm run build" did not complete successfully: exit code: 1
```

Step 3, multi-stage (node:20 build, node:20-alpine run, user node): 193 MB. ls node_modules fails and /health still shows hasBuildManifest: true. Tagged v1.

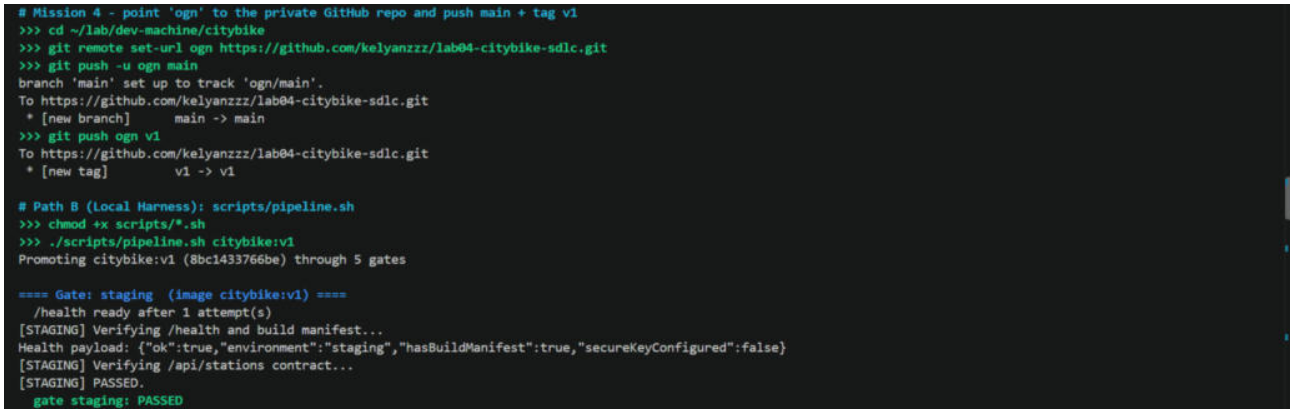


Dockerfile	Image size	Rebuild after 1-line change
Naive	2.04 GB	118 s
Cache fixed	1.85 GB	2.3 s
Multi-stage	193 MB	2.6 s

- A) With package files and npm ci first, a code change does not touch that layer, so npm ci stays cached.
- B) server.js only uses Node built-in modules. It needs public/ and the manifest, copied from the build stage, so hasBuildManifest is true.
- C) Security: fewer packages and vulnerabilities, non-root user. Ops: smaller image, faster pull and rollback.

Mission 4

ogn now points to the GitHub repo; main and tag v1 are pushed. scripts/pipeline.sh promotes the same image through the 5 gates (gates on port 4174, prod on 4173).



All five gates pass for citybike:v1. The pipeline is green and citybike-prod stays running.

```
==== Gate: uat  (image citybike:v1) ====
  /health ready after 1 attempt(s)
[UAT] Verifying UI static assets (/, /app.js, /styles.css)...
[UAT] Verifying 4 stations and Riverside Park at 0 bikes...
[UAT] Scenario 1: Valid reservation at Central Square (8 bikes)...
[UAT] Scenario 2: Reject reservation with invalid email format...
[UAT] Scenario 3: Reject reservation at zero-bike station (Riverside Park)...
[UAT] PASSED.
  gate uat: PASSED

==== Gate: oat  (image citybike:v1) ====
  /health ready after 1 attempt(s)
[OAT] Testing malformed JSON handling (expect HTTP 400)...
[OAT] Testing >10KB payload flood guard...
[OAT] Running 50-request burst load test...
[OAT] PASSED.
  gate oat: PASSED

==== Gate: preprod  (image citybike:v1) ====
  /health ready after 1 attempt(s)
[PRE-PROD] Auditing Docker image size for citybike:v1...
[PRE-PROD] Verifying devDependencies (typescript) are excluded from runtime image...
[PRE-PROD] Verifying environment promotion flag (APP_ENV=preprod) and PROD_API_KEY...
[PRE-PROD] PASSED.
  gate preprod: PASSED

==== Gate: prod  (image citybike:v1) ====
  /health ready after 1 attempt(s)
[PROD] Running post-deployment production smoke & secret verification...
[PROD] Production verification PASSED.
  gate prod: PASSED

PIPELINE GREEN: citybike:v1 is live in production on port 4173
>>> git add . && git commit -m 'ci: five-gate promotion pipeline (scripts/pipeline.sh)' && git push
[main d5cf055] ci: five-gate promotion pipeline (scripts/pipeline.sh)
 6 files changed, 88 insertions(+)
 create mode 100755 scripts/pipeline.sh
 mode change 100644 => 100755 scripts/test-oat.sh
 mode change 100644 => 100755 scripts/test-preprod.sh
 mode change 100644 => 100755 scripts/test-prod.sh
 mode change 100644 => 100755 scripts/test-staging.sh
 mode change 100644 => 100755 scripts/test-uat.sh
To https://github.com/kelyanzzz/lab04-citybike-sdlc.git
   dff5a1e..d5cf055  main -> main
 *  Terminal will be reused by tasks, press any key to close it.
```

- A) Each gate depends on the previous one. Stopping at the first failure keeps a bad image away from prod.
- B) Staging: it starts and the API works. UAT: business rules. OAT: bad input and load. Preprod: size, no devDependencies, prod config. Prod: real secret.
- C) Build once so the tested image is exactly the deployed image. Only the configuration changes.
- D) Without PROD_API_KEY the app uses an insecure default key and still looks healthy. Preprod and Prod catch it.

Mission 5

Part 1, no gate: v2 works on my machine with DB_URL set, then goes straight to prod.

```
# Mission 5 part 1 - Terminal A (Dev): v2 'feature', no gate
>>> cd ~/lab/dev-machine/citybike
>>> DB_URL=postgres://localhost:5432/app npm start

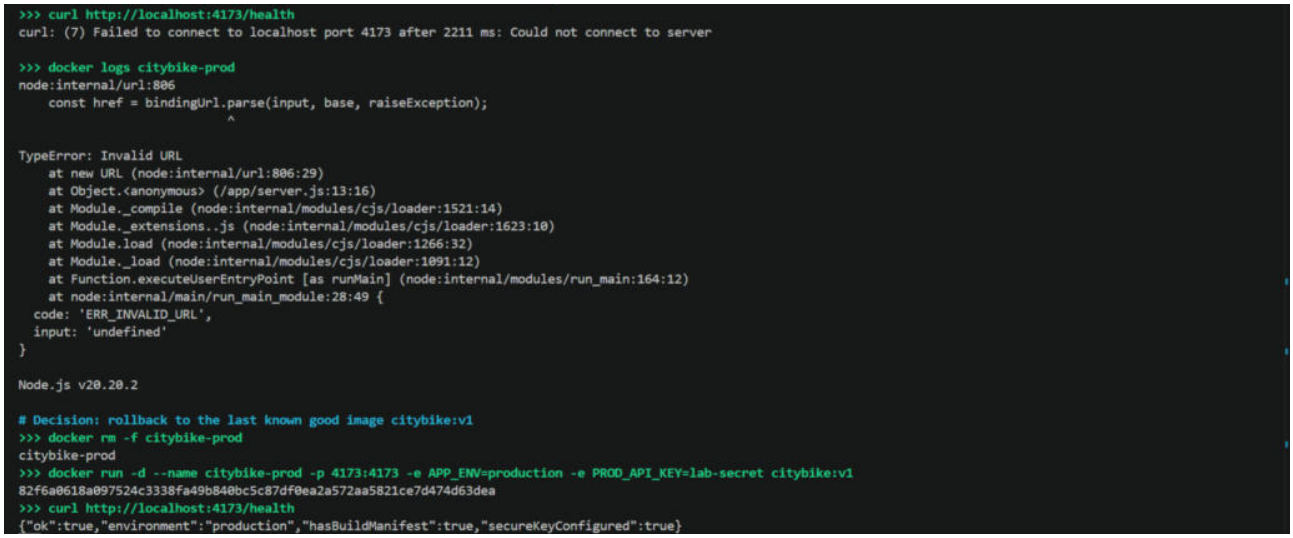
> citybike-sdlc@1.0.1 start
> node server.js

db host: localhost
CityBike service listening on http://0.0.0.0:4173 [env=development]
```

```
>>> docker rm -f citybike-prod
citybike-prod
>>> docker run -d --name citybike-prod -p 4173:4173 -e APP_ENV=production -e PROD_API_KEY=lab-secret citybike:v2
d91f0e1ffce883432f777c1afcf1888e9e0c3be41e174843eede91f314dabeca
```

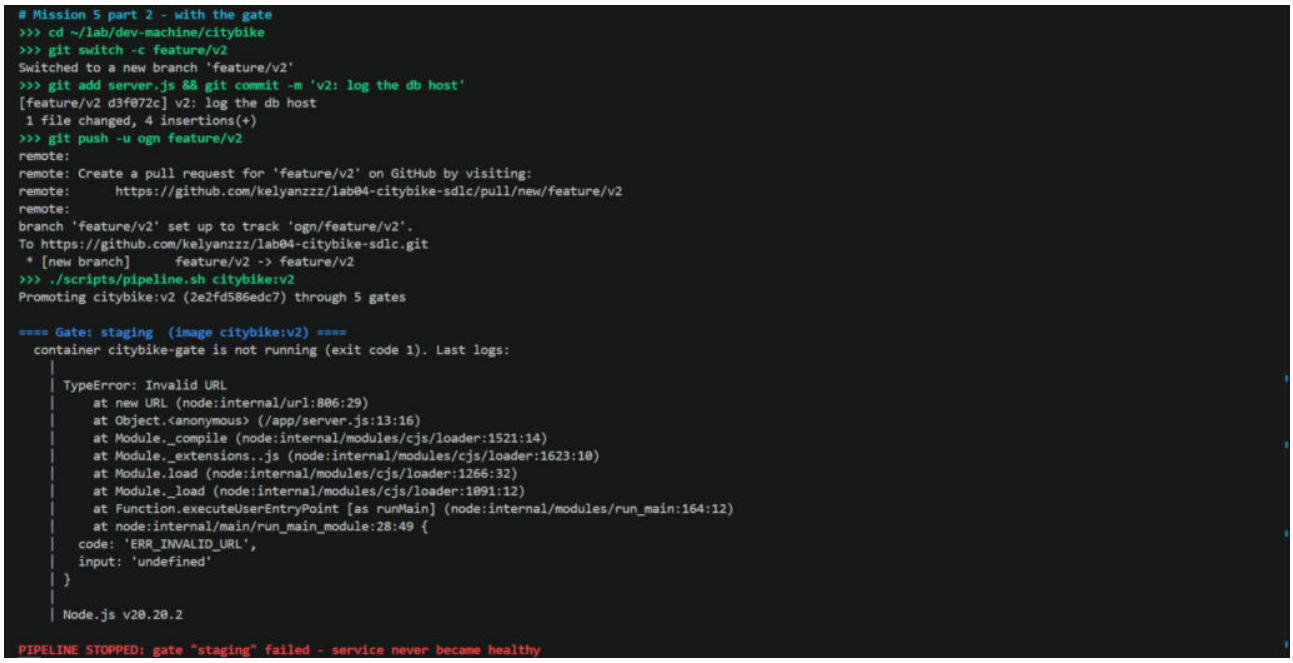
QA: the prod container exited, /health does not answer, the logs show TypeError: Invalid URL (DB_URL is missing in prod).

```
# Mission 5 - Terminal B (QA): assess the outage
>>> docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS                     PORTS
                                                                                                                                NAMES
d91f0e1ffce8   citybike:v2                          "docker-entrypoint.s…"   36 seconds ago   Exited (1) 35 seconds ago
```



Decision: rollback. Prod was down and citybike:v1 is a known good image. Rollback takes seconds; a fix needs a new build and all the gates. After the rollback /health is ok again.

Part 2, with the gate: v2 on feature/v2, pushed, then the pipeline. It stops at Staging and prod is never touched.



- When a gate is red: stop, keep prod on the last good version, fix and push again.
- Why practice the rollback: so it is fast and sure to work during a real outage.

Mission 6

Commit history: the starter commits, then my commits for each mission, and the feature/v2 branch.

```
# Mission 6 - Deliverables: git commit history
>>> cd ~/lab/dev-machine/citybike
>>> git log --oneline --graph --all
* d3f072c v2: log the db host
* d5cf055 ci: five-gate promotion pipeline (scripts/pipeline.sh)
* dff5a1e Dockerfile: cache + multi-stage
* c4e1f75 Dockerfile: cache-friendly layer order + .dockerignore
* 301db61 Dockerfile: naive single-stage
* af9dc4d v1: station snapshot
* fa95935 docs; update README
* 10268b4 test: cover secure key configured
* 976fcd1 test: cover 4-station count check in UAT
* 22323aa ci: add uat testing
* bd8c0b4 refactor: update gitignore
* 540004f fix: switch module system to commonjs
* 1179a0f feat(public): add static files
* 6c56529 docs(readme): add lab instructions
* b912e96 initial commit
```